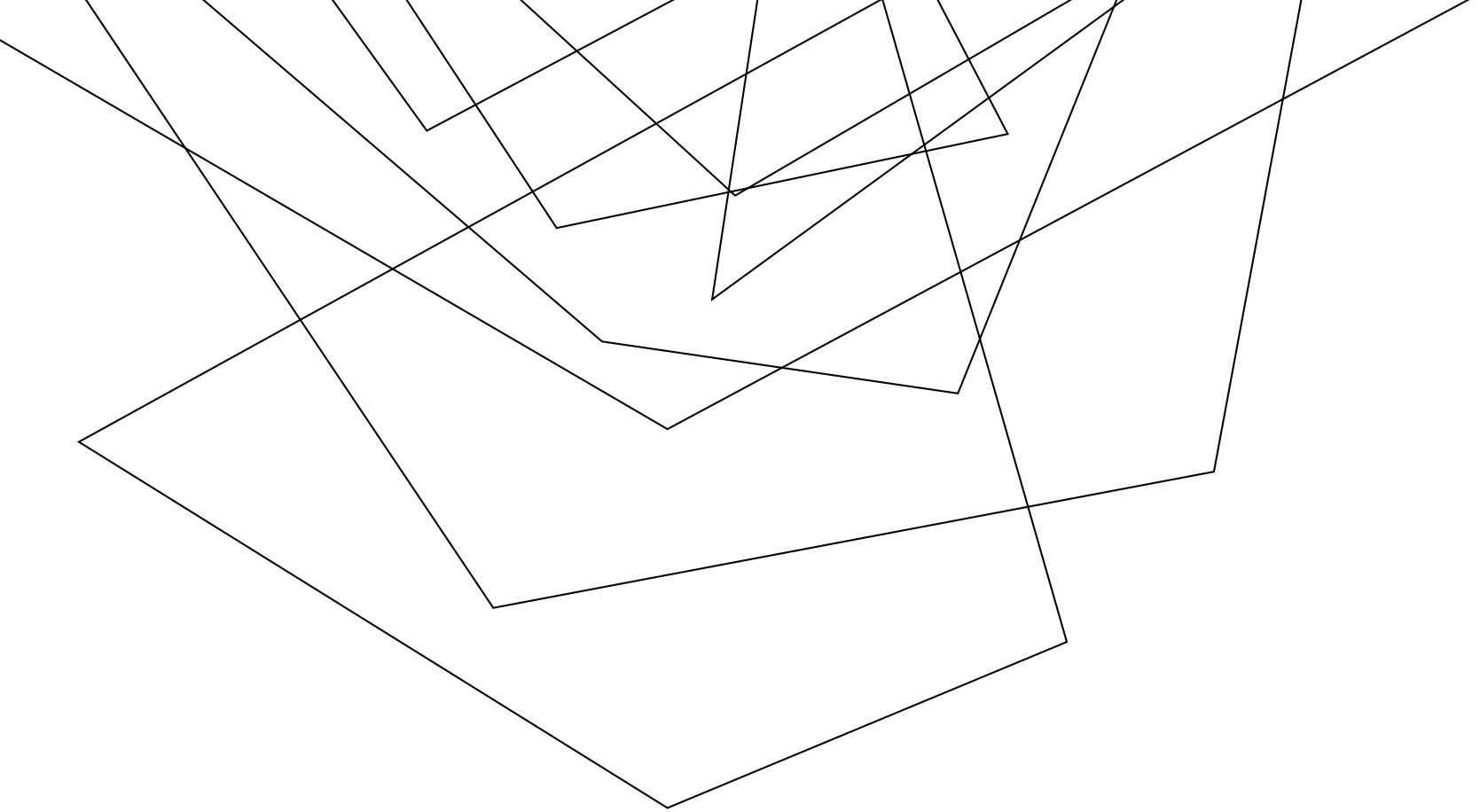


Hof University
University of Applied Sciences

APPLIED ARTIFICIAL INTELLIGENCE

06 – RECURRENT NEURAL NETWORKS (RNNs)

PROF. DR. LARS ACKERMANN-IGL

MOTIVATION: A COMMONALITY BETWEEN CLASSIFICATION AND REGRESSION PROBLEMS

(IMAGE) CLASSIFICATION TASK

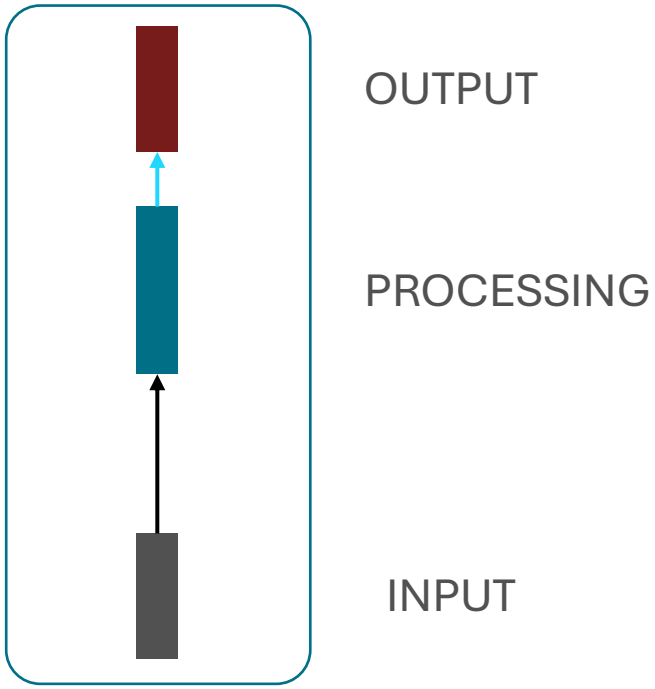
cat

REGRESSION

17.3

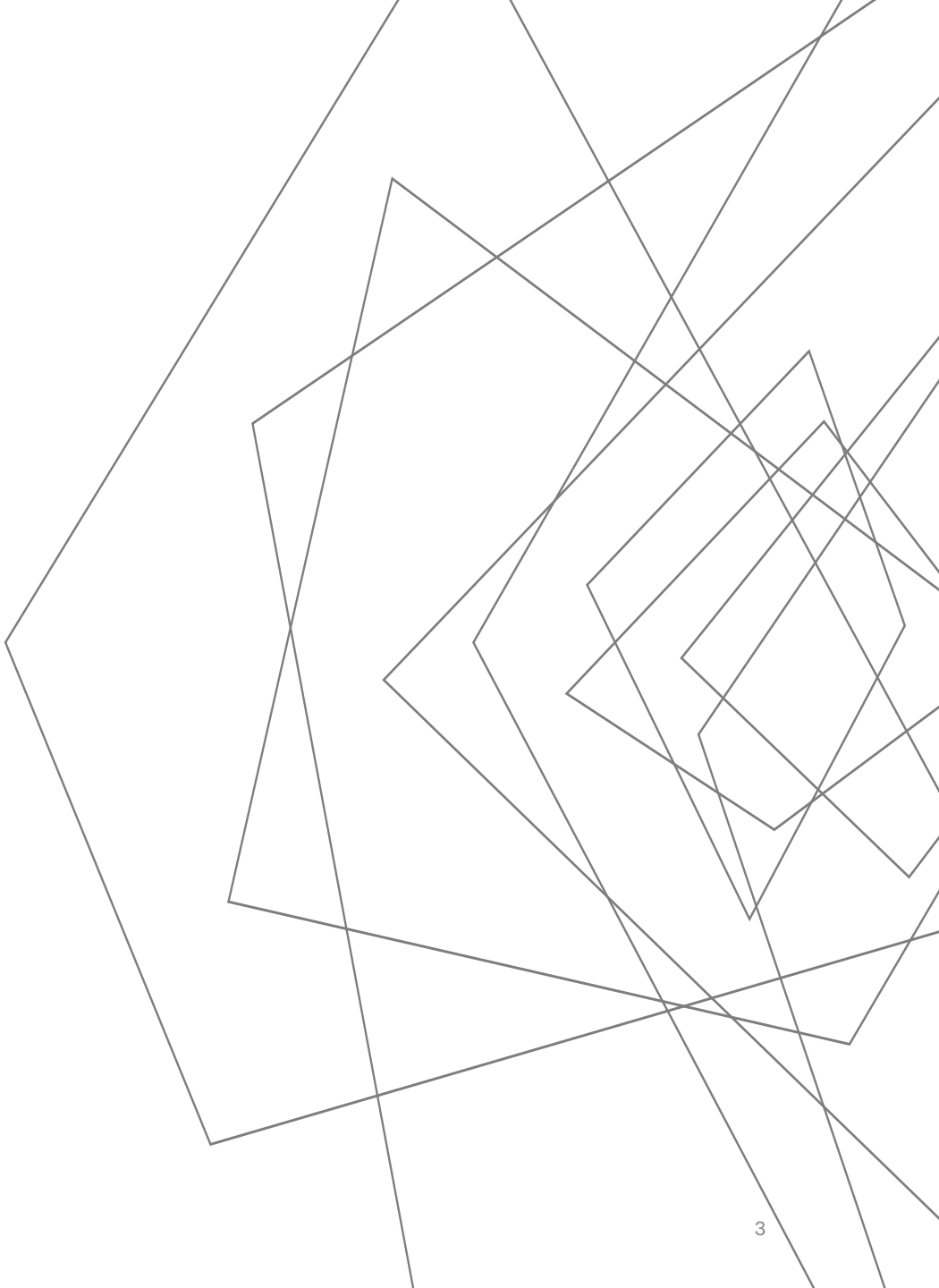
$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ y_4 \\ \dots \end{bmatrix}$$

"SINGLE INPUT SINGLE OUTPUT"



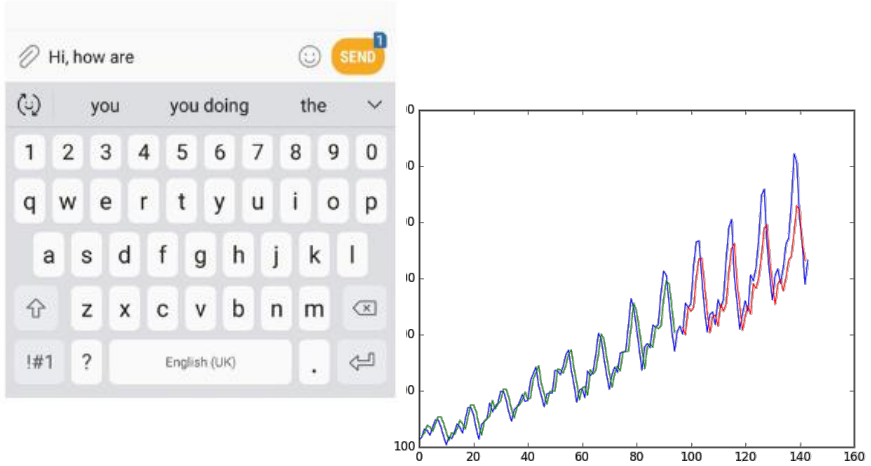
PROBLEMS WITH SEQUENTIAL INPUT

- Sequence processing task
- Application scenarios

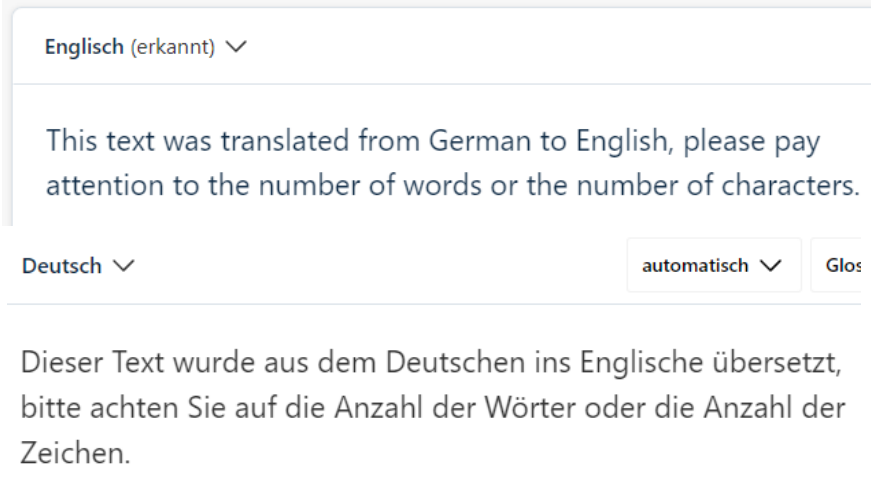


MOTIVATION: PROBLEMS WITH DIFFERING INPUTs

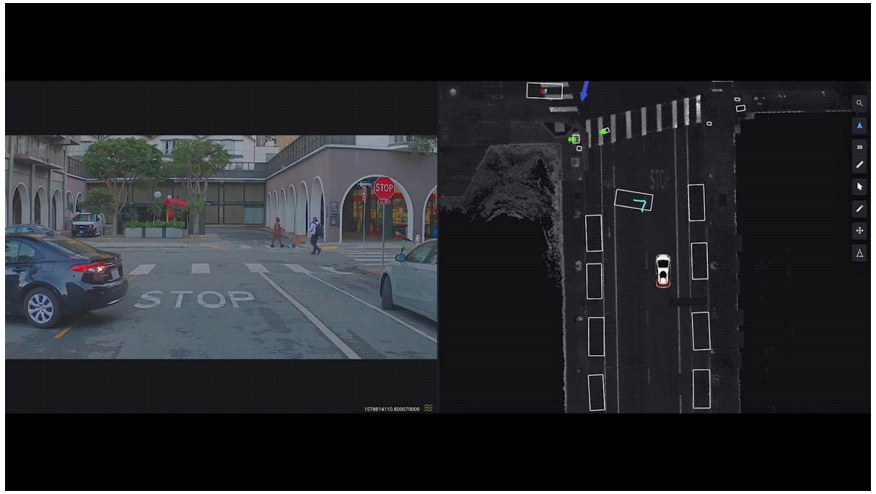
(Time-series) Prediction



Translation



Real-time processing



Captioning

"man in black shirt is playing guitar."

"construction worker in orange safety vest is working on road."

MOTIVATION: WHY IS SEQUENTIAL DATA SPECIAL?

Example: Stock price prediction

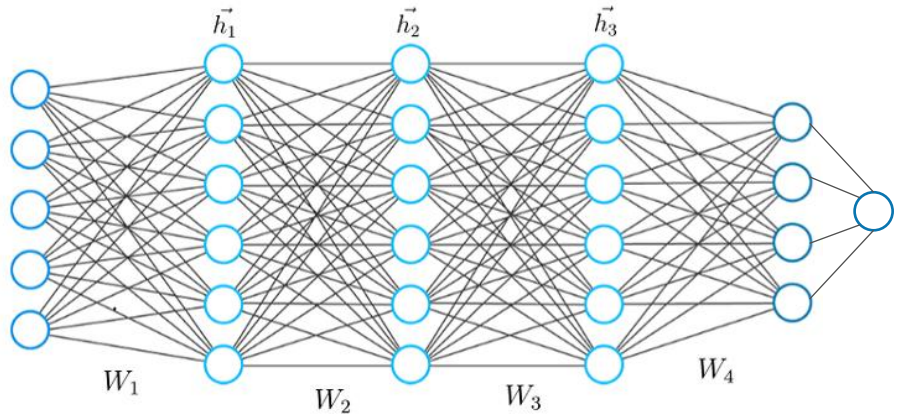
What

Example: "Next best token" prediction

Input		Output
The last BOOM party was	→	insane

vs

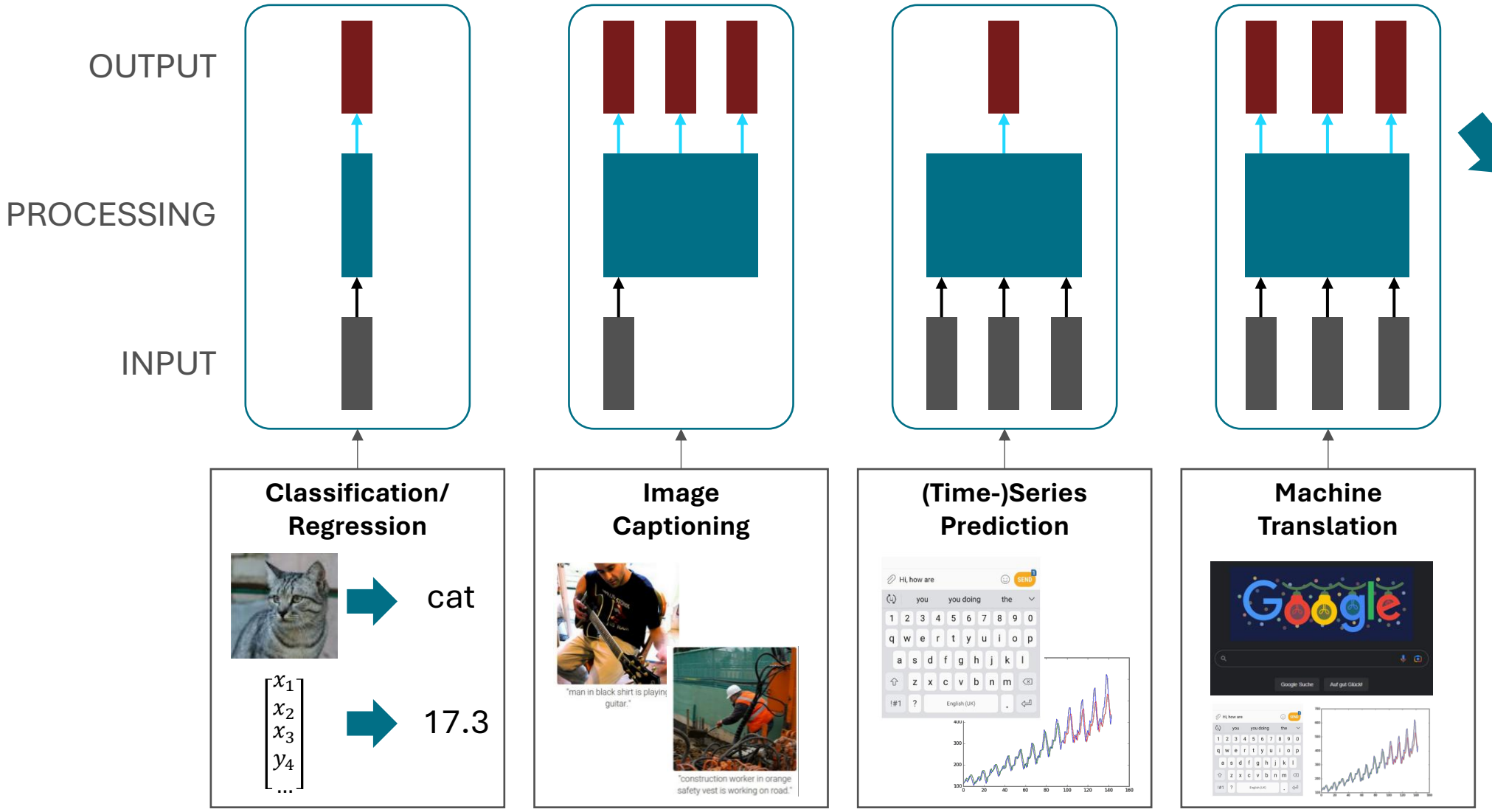
The last BOOM party was simply	→	awesome



Question

Is a differing input length an issue for our neural networks?

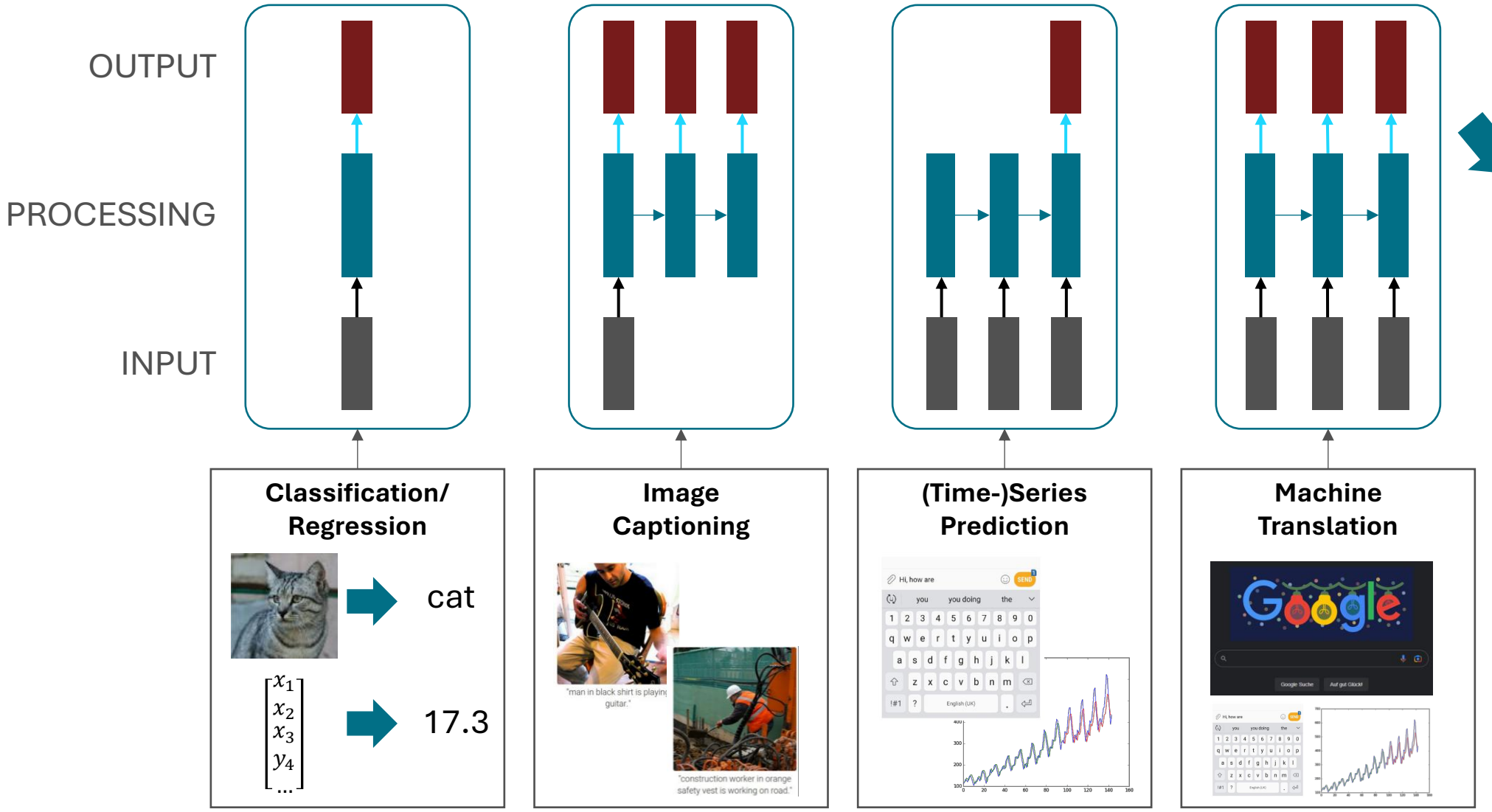
WHEN THE INPUT IS NOT ENOUGH – OR: WORKING WITH SEQUENTIAL DATA



Observation:
All tasks with a "many" operation include some dependencies

Idea:
Include also information from former time-steps into current calculations

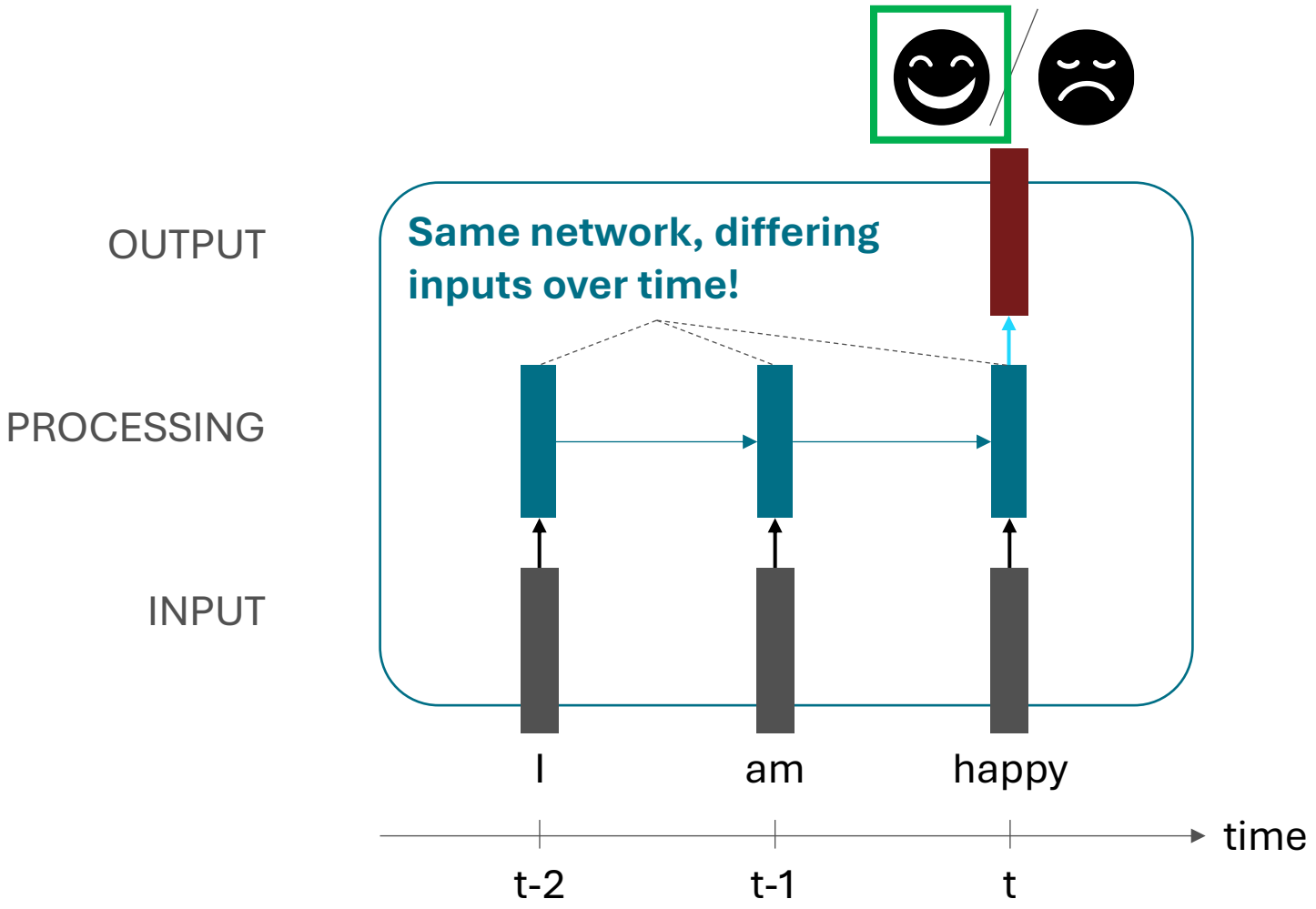
WHEN THE INPUT IS NOT ENOUGH – OR: WORKING WITH SEQUENTIAL DATA



Observation:
All tasks with a "many" operation include some dependencies

Idea:
Include also information from former time-steps into current calculations

WHAT THE DEPICTION MEANS

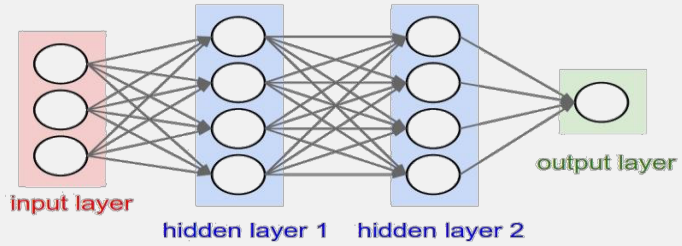


Question

So, if this depiction does NOT show an architecture but instead visualizes how data is processed over time … how does the architecture then look like?

MOTIVATION: ARCHITECTURES LEARNED SO FAR

Multi-layer Perceptron (MLP)



Consists of **one input layer**, **multiple (fully-connected) hidden layers**, and **one output layer (all fixed-sized)**

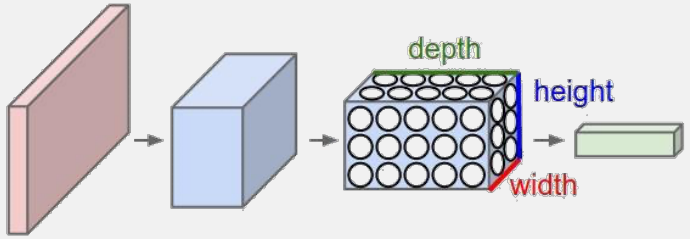
Processing of an **input vector** (feature vector)

Typically usage of **hand-crafted / manually selected features**

Massive amount of trainable parameters

Typically **not suitable** for input information in form of **images** or **sequence data**

Convolutional Neural Network (CNN)



Consists of **one input layer**, **multiple hidden layers** (different types, e.g. CONV, POOL, FC layers), and **one output layer (all fixed-sized)**

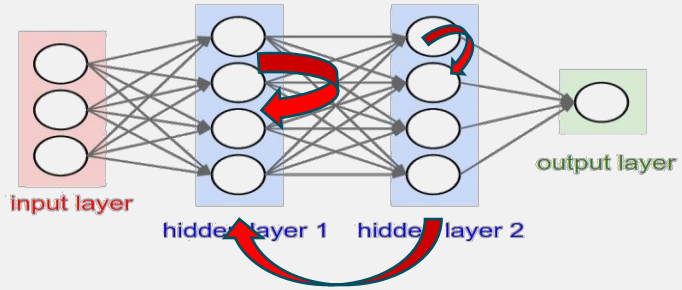
Processing of an **input volume** (image)

Typically utilizing **automatic feature learning** (done by convolutions)

Utilizing concepts for **parameter reduction**

Typically **suitable** for input in form of **images** but not suitable for **sequence data**

?



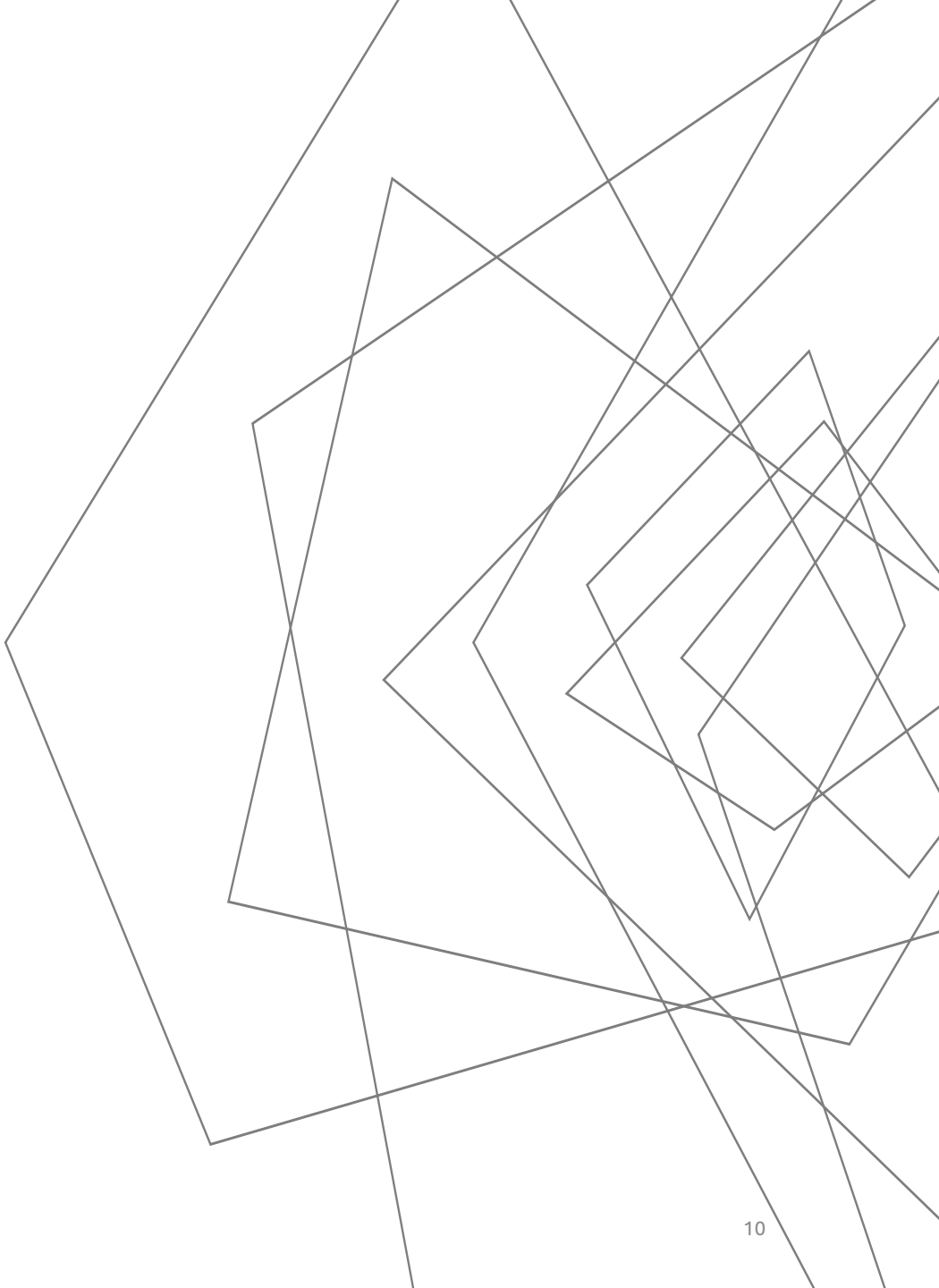
The architectures we talk about recently (MLPs, CNNs) are both variants of **feed-forward NNs**, therefore it is **only allowed** to have **connections between layer l and l+1**.

When we think about a **generic graph structure**, there might be **self-references, circles, backward connections** etc.

Since non of our recently utilized approach is able to deal with sequence data, or variable-sized in-/outputs, we might be able to utilize a more generic graph structure in order to deal with this.

RECURRENT NEURAL NETWORKS (RNNs)

- Basics
- Elman Networks (aka: Simple RNNs)



RNN BASICS

IDEA

To **include information from former processing steps** into current calculation, utilize an **internal (hidden) state per neuron** (in terms of RNNs typically called **cell**), which is **continuously updated** while a sequence is processed.

We can process a sequence of vectors x by applying a recurrence formula at every time step:

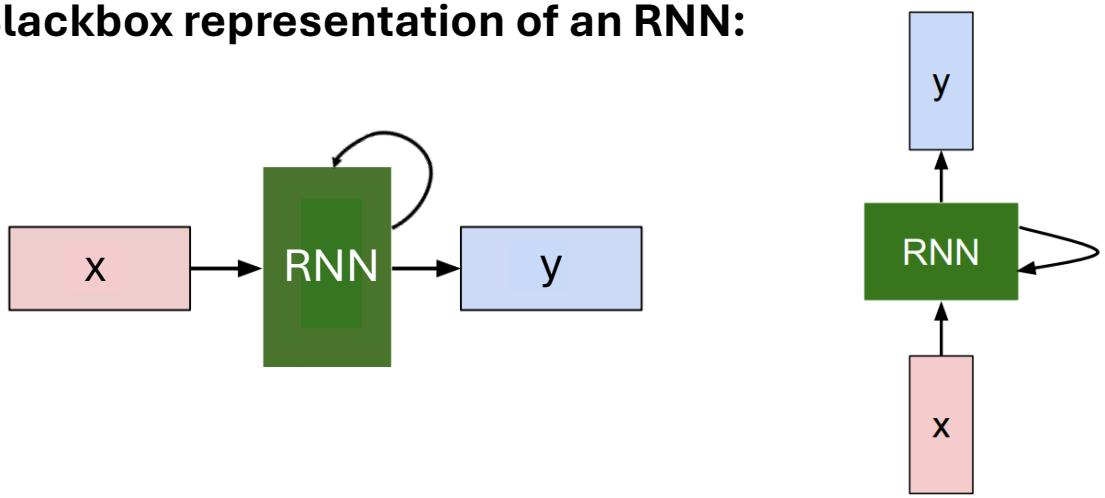
$$h_t = f_W(h_{t-1}, x_t)$$

new state — some function with parameters W — old state — input vector at some time step

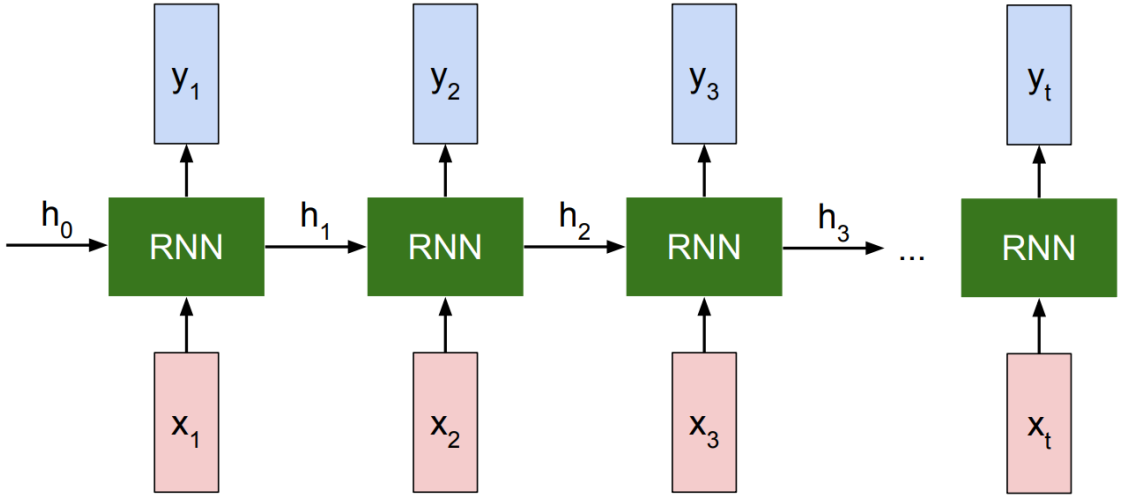
Question

How can f_w and the hidden state look like?
How many parameters do we have to train?
How can the output be calculated?

Blackbox representation of an RNN:



Unrolled representation of an RNN:



ELMAN NETS – FORWARD PASS/INFERENCE

IDEA

Apply the **same function** and the **same set of weight parameters at every time step**.

Calculate the **output** based on the **hidden state** of a specific neuron / RNN cell.

Elman RNN (also called simple or vanilla RNN):

In Elman RNN cells, there is typically a **linear combination of the available information** (hidden state+input), and a **tanh activation** used.

Elman RNN – Forward pass/inference

$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

Note: Usually there is additionally a bias parameter!

Question 1
How does this computation then work step by step?

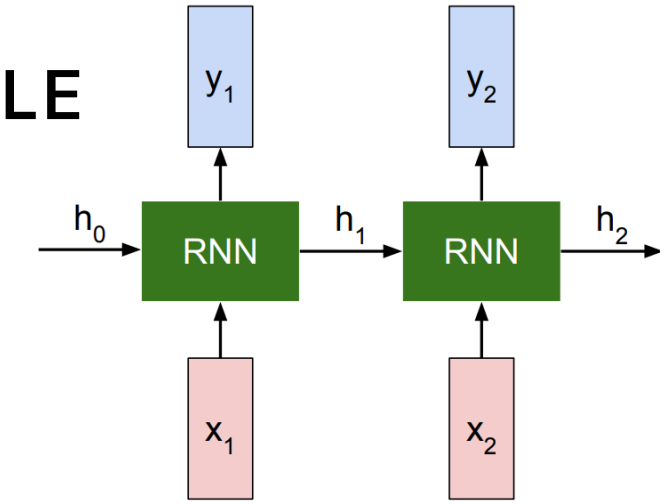
See next slide

Question 2
How does W_{hh}, W_{xh} and W_{hy} look like?

See “Calculating the number of trainable weights”

Question 1
How does this computation then work step for step?

ELMAN NETS: VERY SIMPLE INFERENCE COMPUTATION EXAMPLE



GIVEN

Inputs

$$x_1 = 4, x_2 = -1$$

Weights and Biases

$$W_{xh} = 1$$

$$W_{hh} = 2$$

$$b_h = -1$$

$$W_{hy} = 3$$

$$b_y = 1$$

Initial hidden state

$$h_0 = 0$$

TASK

Compute output y at t=2

Forward pass $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$

$y_t = W_{hy}h_t + b_y$

tanh $f(x) = \frac{(e^x - e^{-x})}{(e^x + e^{-x})}$

INFERENCE COMPUTATION

Hidden states

Time step 1: $x_1 = 4$

$$h_1 = \tanh(W_{xh} \cdot x_1 + W_{hh} \cdot h_0 + b_h) = \tanh(1 \cdot 4 + 2 \cdot 0 - 1) = \tanh(3) \approx 0.995$$

Time step 2: $x_2 = -1$

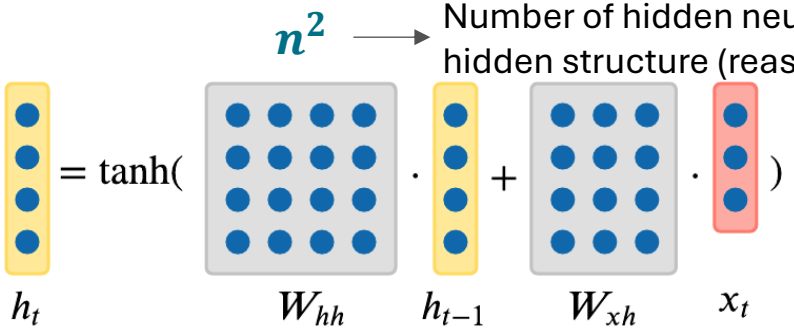
$$h_2 = \tanh(W_{xh} \cdot x_2 + W_{hh} \cdot h_1 + b_h) = \tanh(-1 + 2 \cdot 0.995 - 1) = \tanh(-1 + 1.99 - 1) = \tanh(-0.01) \approx -0.01$$

Output at t=2

$$y_2 = W_{hy} \cdot h_2 + b_y = 3 \cdot (-0.01) + 1 = -0.03 + 1 = 0.97$$

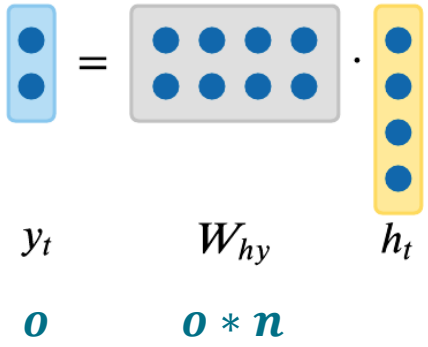
Question 2
How does W_{hh}, W_{xh} and W_{hy} look like?

ELMAN NETS: CALCULATING THE NUMBER OF TRAINABLE WEIGHTS



Number of hidden neurons n for which the output is connected with the same hidden structure (reason for squaring n)

Number of input neurons m is determined by the dimensionality of the input (e.g., the number of features or the size of the vocabulary)

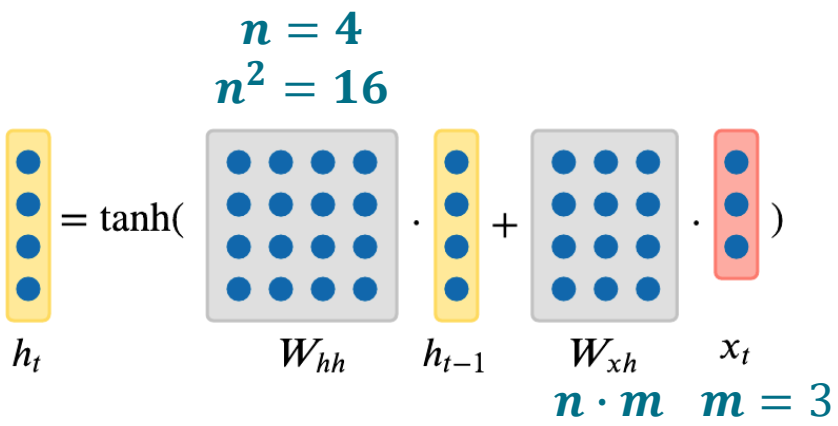


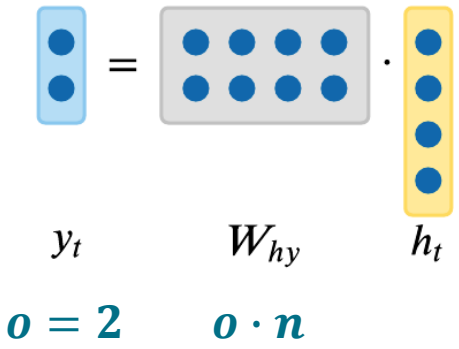
Number of output neurons o is determined by the dimensionality of the output (e.g., the number of features or the size of the vocabulary)

NUMBER OF TRAINABLE WEIGHTS

$$N_{params} = n^2 + n * m + o * n$$

ELMAN NETS: CALCULATING THE NUMBER OF TRAINABLE WEIGHTS





NUMBER OF TRAINABLE WEIGHTS

$$N_{params} = n^2 + n * m + o * n$$

DECOMPOSING THE EXAMPLE

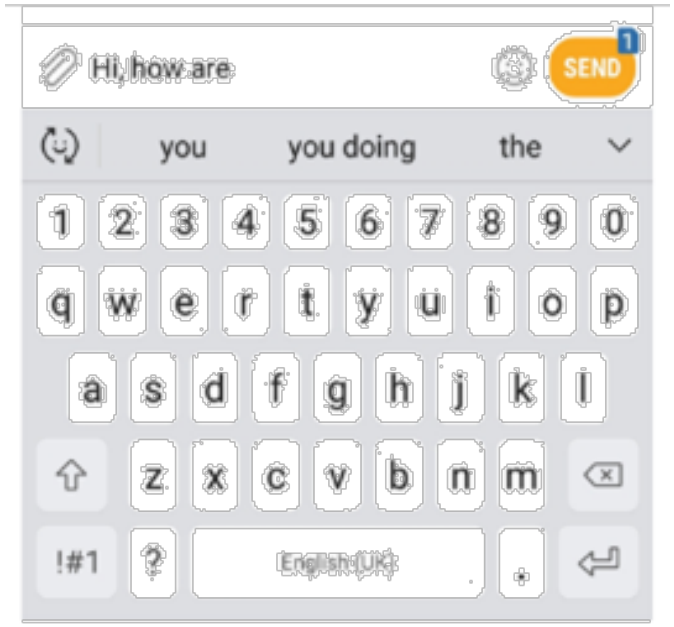
Define a simple RNN, which takes a **3-dimensional feature vector** at each time-step. Hidden state might be represented by **four elements**. For the output, we might calculate **two elements** (e.g. 1-hot class information).
Thus, the number of trainable weights (not considering any bias terms) can be computed as follows:

$$N_{params} = 16 + 12 + 8 = 36$$

ELMAN NETS – NEXT CHARACTER PREDICTION EXAMPLE

EXAMPLE

We have already learned that one application of RNNs is e.g. next word prediction.



We want to start one step smaller, where we want to predict the next character given a sequence of characters.

We assume that there is a vocabulary, consisting of **four elements**:

$$V \in \{\text{"h"}, \text{"e"}, \text{"l"}, \text{"o"}\}$$

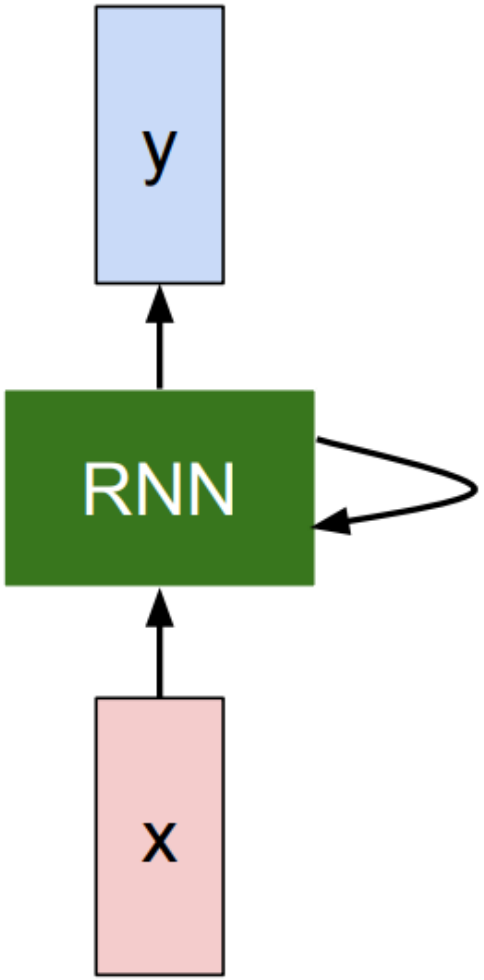
We represent each element of the vocabulary as **one-hot encoded vector**, therefore the **input** and **output size** of the RNN is of **dimension 4x1**.

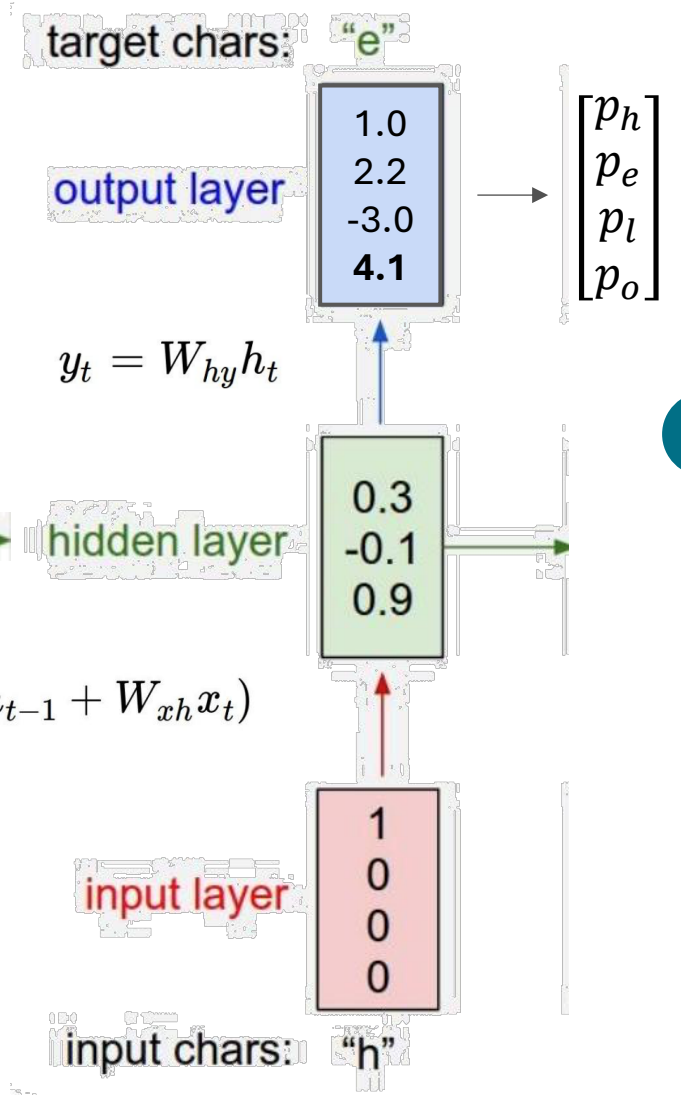
$$\begin{bmatrix}1\\0\\0\\0\end{bmatrix} = \text{"h"} \quad \begin{bmatrix}0\\1\\0\\0\end{bmatrix} = \text{"e"} \quad \begin{bmatrix}0\\0\\1\\0\end{bmatrix} = \text{"l"} \quad \begin{bmatrix}0\\0\\0\\1\end{bmatrix} = \text{"o"}$$

For the initial training, we have only **one sequence „hello“** available.

For the initial training, we apply **some initial / random values for the internal state (e.g., represented by a 3x1 vector), as well as for the weight parameters**.

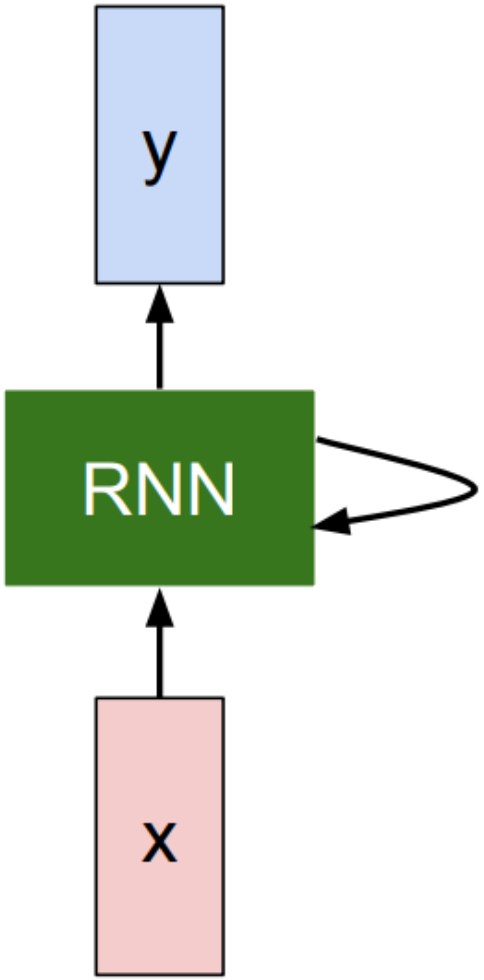
ELMAN NETS – NEXT CHARACTER PREDICTION EXAMPLE

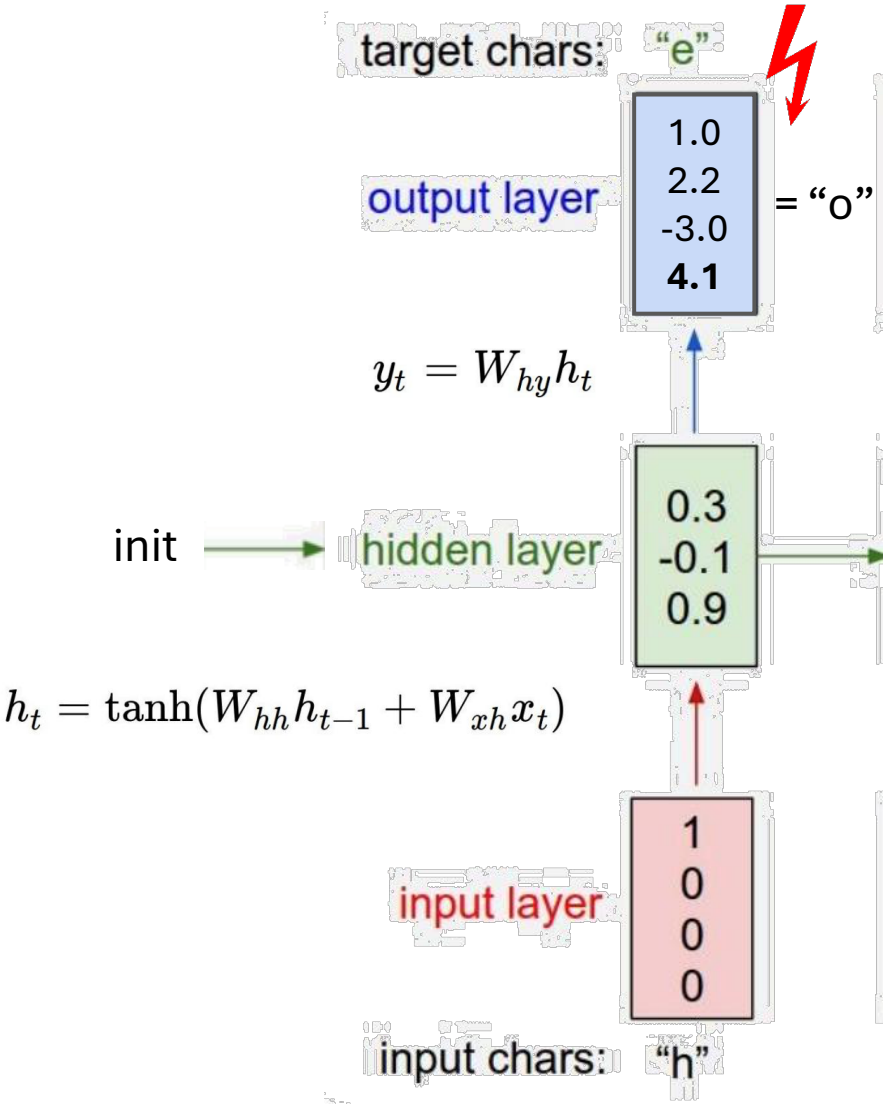




! The calculated hidden state is a summarization of all seen characters until a specific time step.

ELMAN NETS – NEXT CHARACTER PREDICTION EXAMPLE

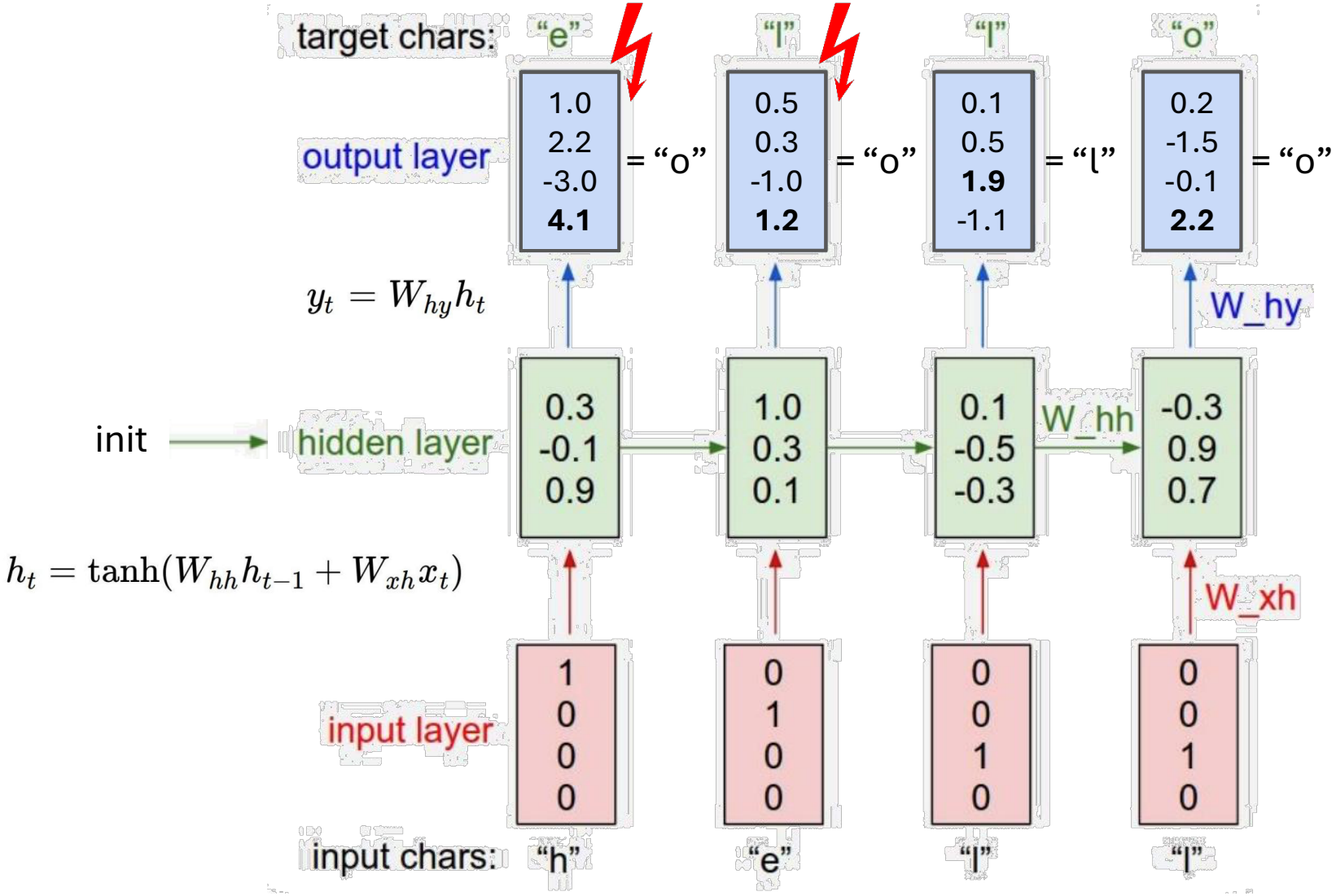




Our currently used parameterization would predict a "o" as next character, but we expect an "e"!
→ **Error / Loss**

The calculated hidden state is a summarization of all seen characters until a specific time step.

ELMAN NETS – NEXT CHARACTER PREDICTION EXAMPLE



As already seen, with our current (initial) parameterization of the weights we can observe some errors at the output layer

→ **We need to adjust / train the parameters**

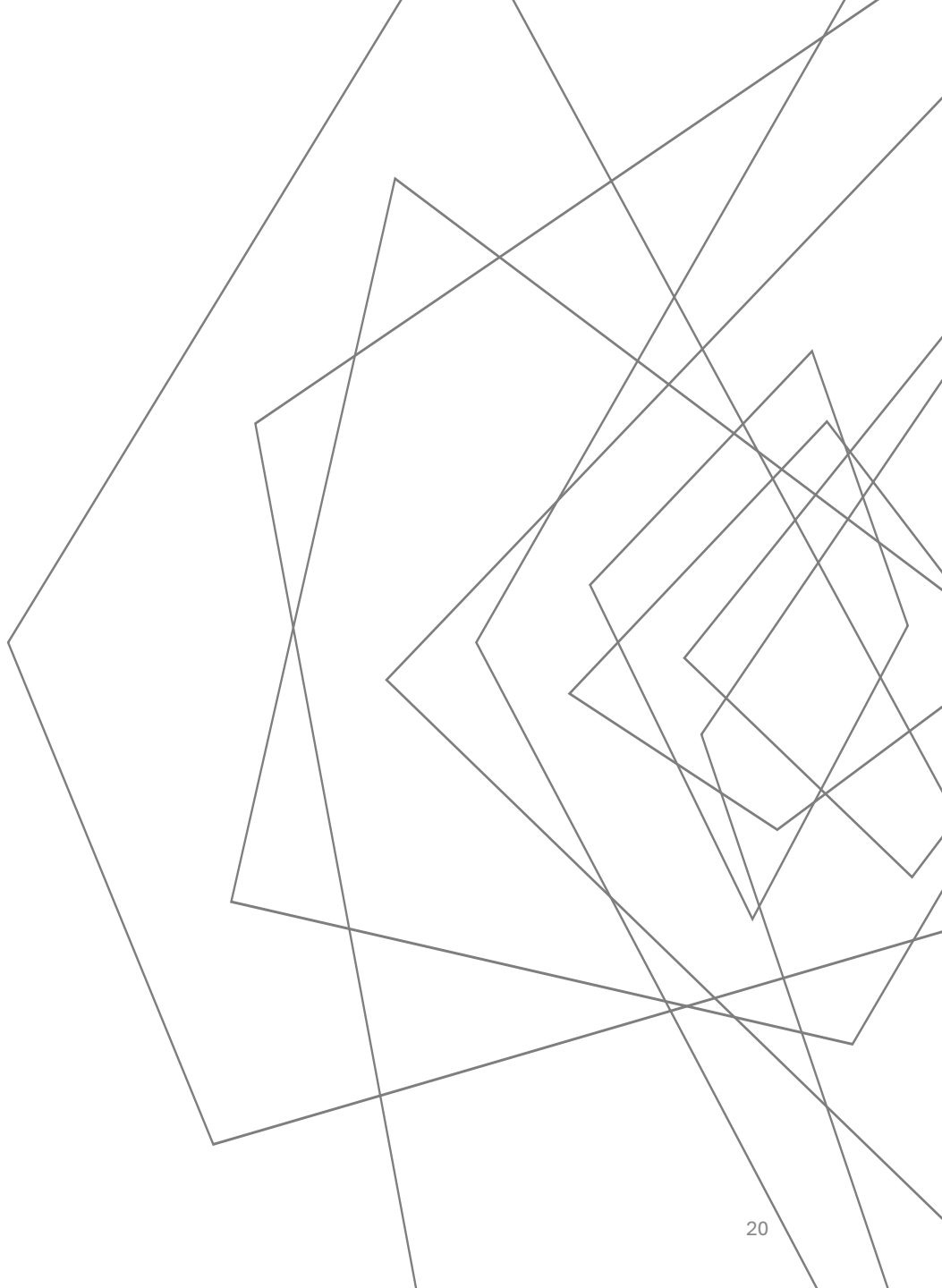
Question

How can we do this?

Hint: Nothing really new here!

TRAINING RNNs

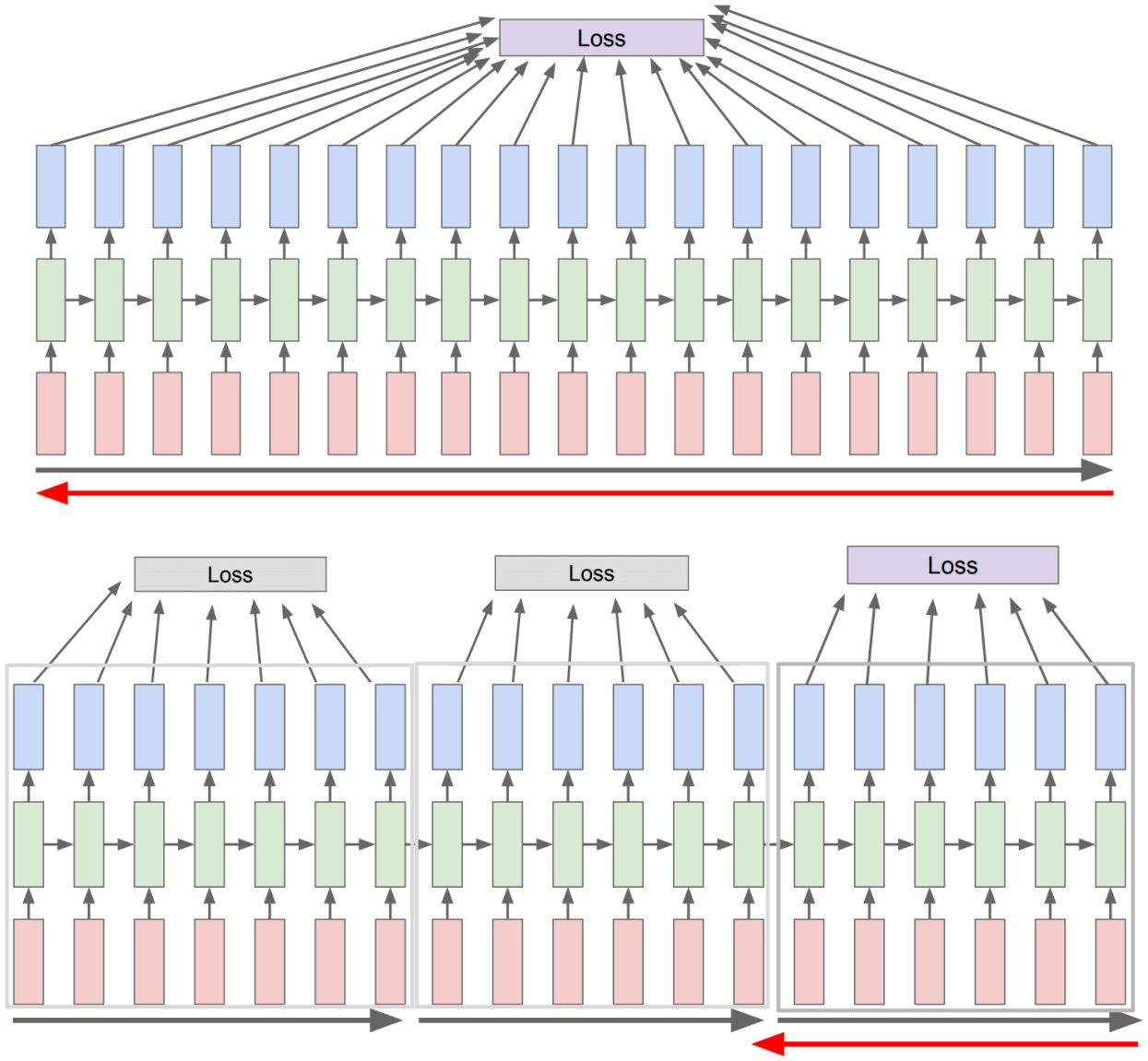
- Backpropagation through time
- Problems with training (gradient flow)
- Solution: Long Short-term Memory Networks (LSTMs)
- Alternative architectures



RECURRENT NEURAL NETWORKS – BACKPROPAGATION THROUGH TIME (BTT)

IDEA

- Training RNNs
  - Use the same general training principle as other neural networks: backpropagation
  - Special variant: **Backpropagation Through Time (BPTT)**
- Backpropagation Through Time
  - Required because RNNs have a temporal (dynamic) component
  - Errors are backpropagated through network layers **AND** across multiple time steps
- Forward Pass
  - Process the entire input sequence
  - Compute the loss **after** the full sequence
- Backward Pass
  - Backpropagate errors through the entire sequence
  - Compute gradients and update parameters
- Truncated BPTT
  - Split long sequences into smaller chunks
  - Commonly used in practice
  - **Justified because simple RNNs struggle with long-term dependencies**

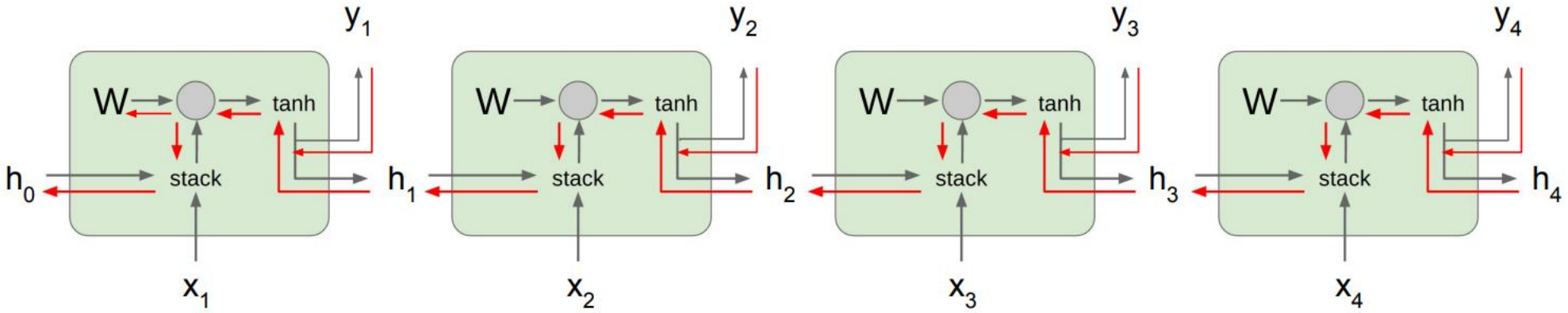


IMPLEMENTING RNNs IN KERAS

- First of all, you need to prepare data similarly to how it is done for the MLPs. However, with RNNs we typically operate on sequential data.
- The example shown here can be used, for instance, to guess the next number in a consecutive sequence of numbers, e.g., 1,2,3 … -> 4.
- Model implementation:
  - SimpleRNN is the class for vanilla RNNs (aka Elman nets, which is why the activation function is tanh by default)
  - units: number of neurons representint the hidden state
  - input_shape: (<number of time steps to process>, <number of features per time step>)
  - return_sequences:
    - if False only returns the last hidden state (that is processed by the Dense layer); used for many-to-one problems
    - If True returns the hidden state for each time step (where each is processed by the Dense layer; used for many-to-many problems
  - Dense layer: Maps the hidden state to the RNN‘s output

```
model = keras.Sequential([
    layers.SimpleRNN(
        units=8,
        input_shape=(3, 1),
        return_sequences=False
    ),
    layers.Dense(1)
])
```

RECURRENT NEURAL NETWORKS – PROBLEMS WHILE TRAINING – GRADIENT FLOW



Problem

When we want to examine how the **output at the very last timestep affects the weights at the very first time step**, we need to calculate the **partial derivative of h_t with respect to h_{t-1}**. We **update the weights W_{hh}** by getting the **derivative of the loss at the very last time step L_t with respect to W_{hh}**. This sums up to the following (math right now not relevant, can be found in [2]):

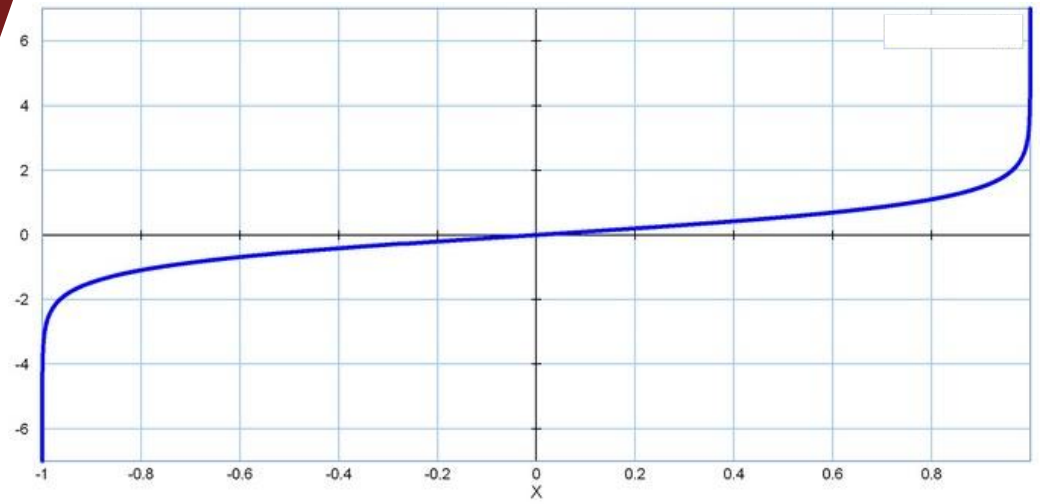
$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = tanh'(W_{hh} h_{t-1} + W_{xh} x_t) W_{hh}$$

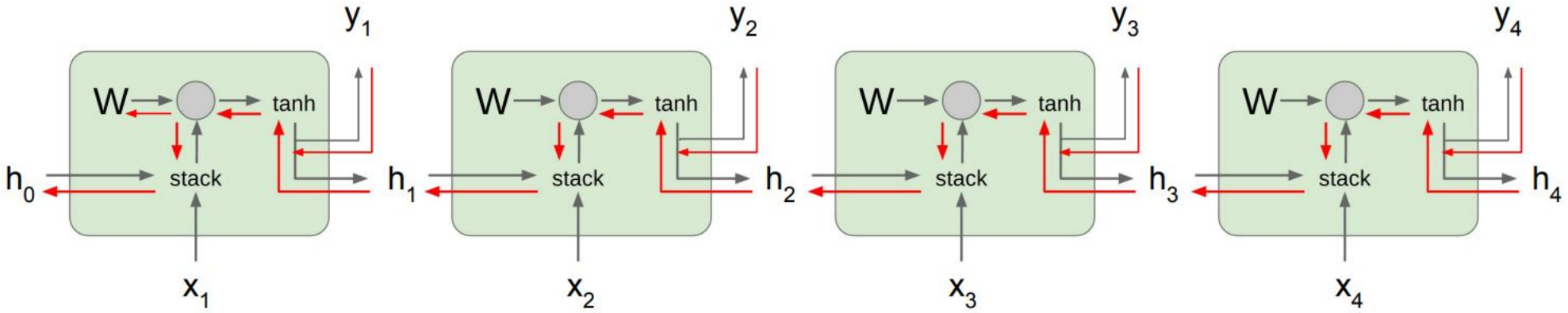
$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}}\right) \frac{\partial h_1}{\partial W}$$

Question

How does this function look like and what impact does it have?



RECURRENT NEURAL NETWORKS – PROBLEMS WHILE TRAINING – GRADIENT FLOW



Problem

When we want to examine how the **output at the very last timestep affects the weights at the very first time step**, we need to calculate the **partial derivative of h_t with respect to h_{t-1}**. We **update the weights W_{hh}** by getting the **derivative of the loss at the very last time step L_t with respect to W_{hh}**. This sums up to the following (math right now not relevant, can be found in [2]):

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = tanh'(W_{hh} h_{t-1} + W_{xh} x_t) W_{hh}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}}\right) \frac{\partial h_1}{\partial W}$$

Question
How does this function look like and what impact does it have?

It will be **very likely**, that the function delivers a **value between 1 and -1**.

Question
What will happen, if we need to apply this function multiple times in order to calculate our gradient?

Vanishing gradient problem
Especially for longer timesteps, gradient tend to decrease in value and get close to value 0. This will be a problem, when we want to model long term dependencies.

$\tanh' = 1 - \tanh^2 x$

EXCURSUS (FOR SELF STUDY): LONG SHORT TERM MEMORY (LSTM)

PROBLEM

There is typically **no possibility** to get rid of the vanishing gradient problem in Simple / Vanilla / Elman RNNs. Therefore, this network architecture is typically **not suitable** if long term dependencies are relevant.

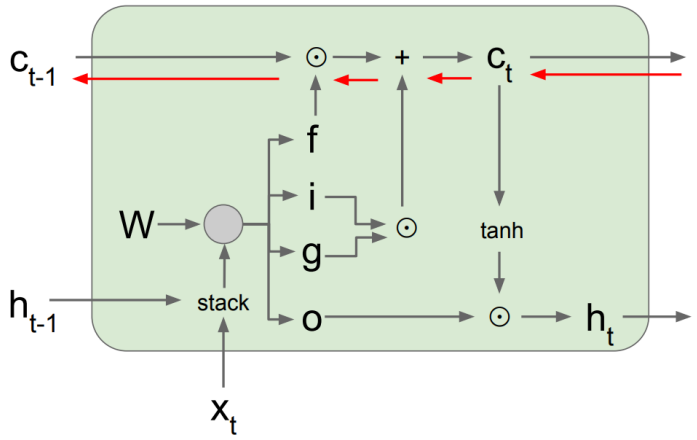
IDEA

Change the architecture of the RNN, that is **has the ability to store relevant long term information**.

This leads to the definition of the **Long Short Term Memory (LSTM)** architecture, which was introduced by Hochreiter & Schmidhuber in 1997.

Besides the **hidden state** introduce an additional **cell state**, which can be used to **store long term information** and can be altered through some special gates:

- **Forget gate**: How much information needs to be removed from the long term memory
- **Input gate**: How much information needs to be added to the long term memory
- **Output gate**: How much information needs to be displayed



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

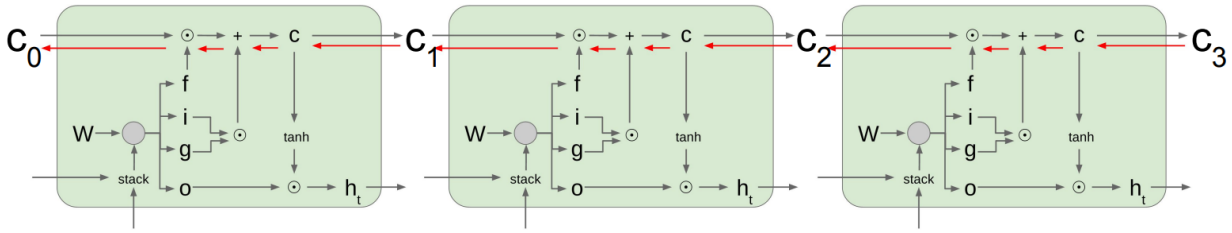
$$h_t = o \odot \tanh(c_t)$$

element-wise Hadamard product

Question

What is the benefit of this additional state?

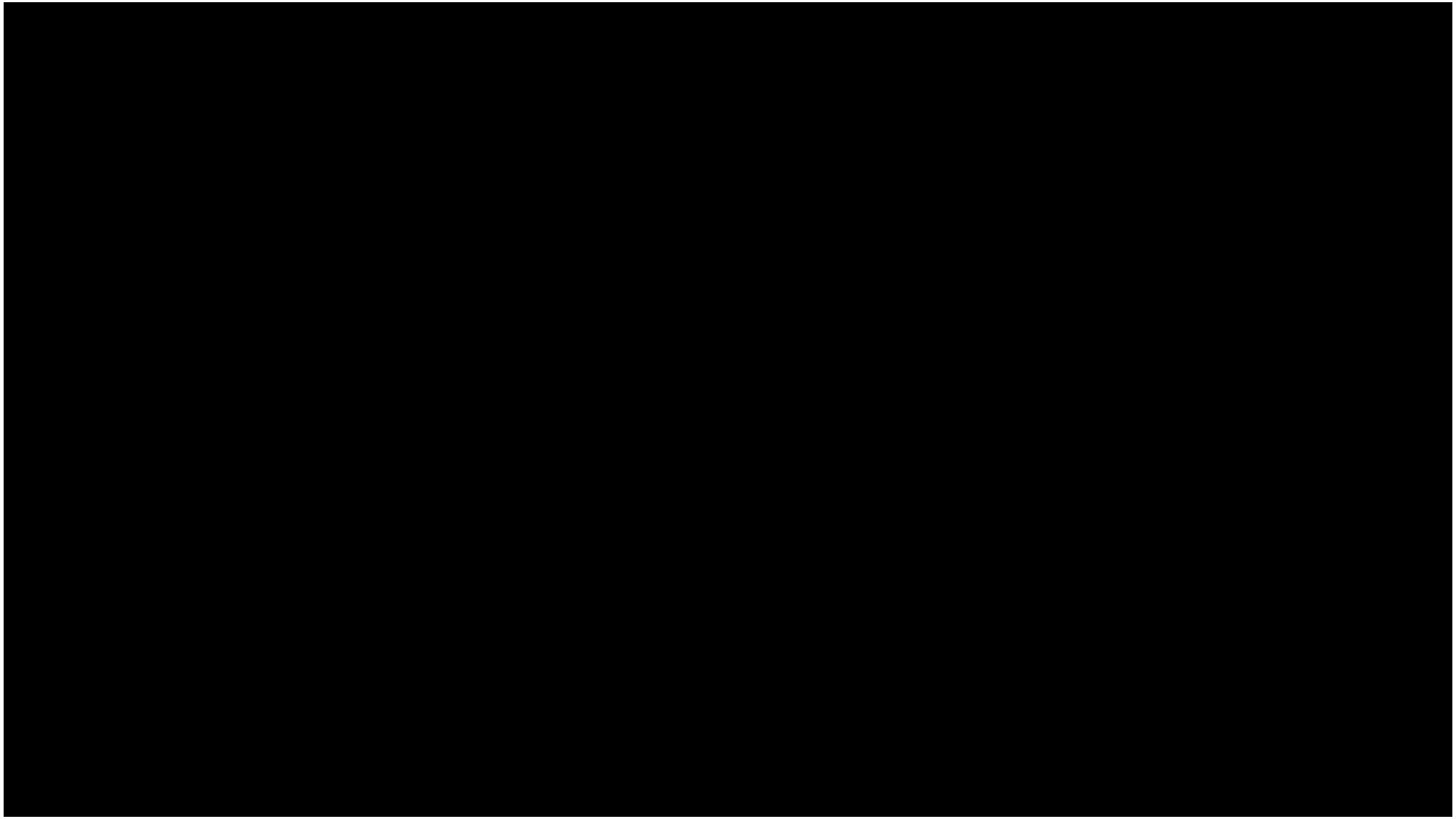
Uninterrupted gradient flow!



\+ easier to model long term dependencies

o no guarantee that VGP does not occur, but its much more unlikely

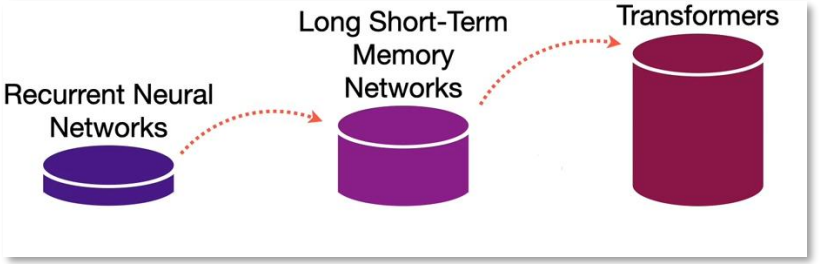
EXCURSUS (FOR SELF STUDY): VIDEO TIME … A FEW MORE WORDS ABOUT LSTMs



https://www.youtube.com/watch?v=b61DPVFX03I

EXCURSUS (FOR SELF STUDY) – ALTERNATIVE ARCHITECTURES

There are a lot more recent architectures which are interesting in the field of sequence processing, but they require some more deep-dive into the field of RNNs, therefore they are out of the focus of our course.



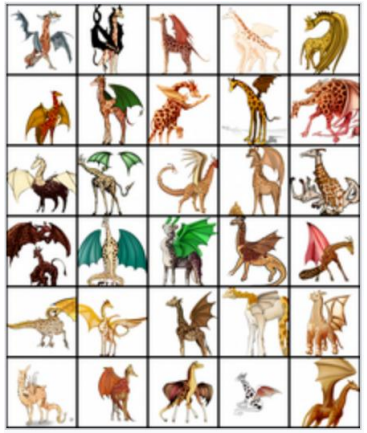
Further architectures are e.g.

- **Gated Recurrent Units (GRU):** Somehow similar to LSTMs, but in general lower amount of parameters.
- **Transformer**: Ability to process sequences in parallel by utilizing a mechanism called attention. Currently state-of-the-art approaches for many problems (e.g. in field of computer vision or natural language processing). Models like **BERT** (Bidirectional Encoder Representations from Transformers), **GPT** (Generative Pre-trained Transformer), **DALL-E**.
- **GPT-2 written article:**

Generative Pre-trained Transformer 2 (GPT-2) is an open-source artificial intelligence created by OpenAI in February 2019.[1][2][3][4][5][6][7][8] GPT-2 translates text, answers questions, summarizes passages,[9] and performs other tasks.[10][11][12] It has been taught to speak English and Japanese , but not the rest of the world 's languages.[13] GPT- 2 is designed with two goals in mind: To recognize patterns in sentences , and To understand complex sentences such as the ones above . It also uses language models that incorporate the principles of gram matical semantics.[14] The first goal is achieved with the use of a deep neural network . This network is built from an array of input modules and is trained by feeding the data into it.

DALL-E:


An image generated by DALL-E 2 based on the text prompt "Teddy bears working on new AI research underwater with 1990s technology"



Images produced by DALL-E 1 when given the text prompt "a professional high quality illustration of a giraffe dragon chimera. a giraffe imitating a dragon. a giraffe made of dragon." (2021)